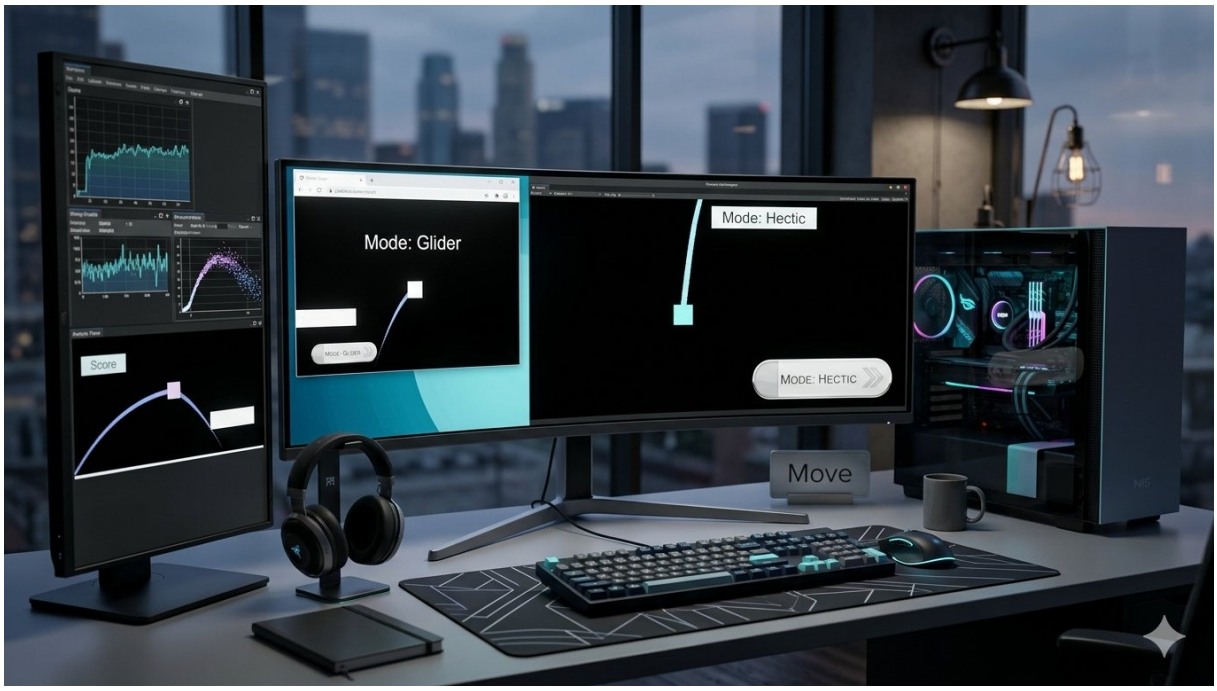


Multi-Move

Sandbox de plataformas 2D en Unity

Eva Gallardo Romero

9 de febrero de 2025

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **3**
3.1 Controles 3

4 Requisitos del sistema **4**
4.1 Requisitos funcionales 4
4.2 Requisitos no funcionales 4

5 Documentación técnica **5**
5.1 MovementStats.cs 5
5.2 Movement.cs 5
5.3 MovementTypes.cs 7
5.4 CameraFollow.cs y ContantRotationReset.cs 8

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: Multi-Move.

El juego se puede probar en el navegador en itch.io y la documentación del código generada con Doxygen está en evagr11.github.io/Multi-Move.

1 Descripción general

Multi-Move es un **sandbox de plataformas 2D** hecho en **Unity** con **C#** en la asignatura de Programación de 1.º de DAM. El jugador controla un cubo que se mueve, salta y puede cambiar en cualquier momento entre **seis modos de movimiento** con la tecla **M**. Cada modo tiene su propia aceleración, velocidad, fricción, saltos y gravedad.

Cada modo es un **ScriptableObject**: un archivo de datos que se edita desde el Inspector de Unity sin tocar el código. Añadir un modo nuevo es crear otro asset y arrastrarlo a la lista.

Modo	**Velocidad**	**Salto**	**Gravedad (sube → pico → cae)**	**Saltos en el aire**
Basic	70	10	2 → 3 → 4	1
Underwater	50	7	0,5 → 0,3 → 0,5	99
Glider	50	8	1 → 1,5 → 2	2
Hectic	120	12	5 → 6 → 8	3
Rocket	60	15	3 → 6 → 10	1
Slippy Flappy	50	7	0,5 → 0,8 → 2	5

El juego da información visual del estado del cubo: cambia de **color** según los saltos que le quedan, las **partículas** cambian de color al subir, en el punto más alto y al caer, y hay partículas al saltar, al aterrizar y cuando ya no se puede saltar más.

2 Diagrama de clases

`Movement` mueve al cubo con las físicas de `Rigidbody2D` usando los valores del `MovementStats` activo. `MovementTypes` guarda los seis modos y, al pulsar M, le pasa el siguiente a `Movement`. `CameraFollow` sigue al jugador y `ContantRotationReset` evita que el cubo gire.

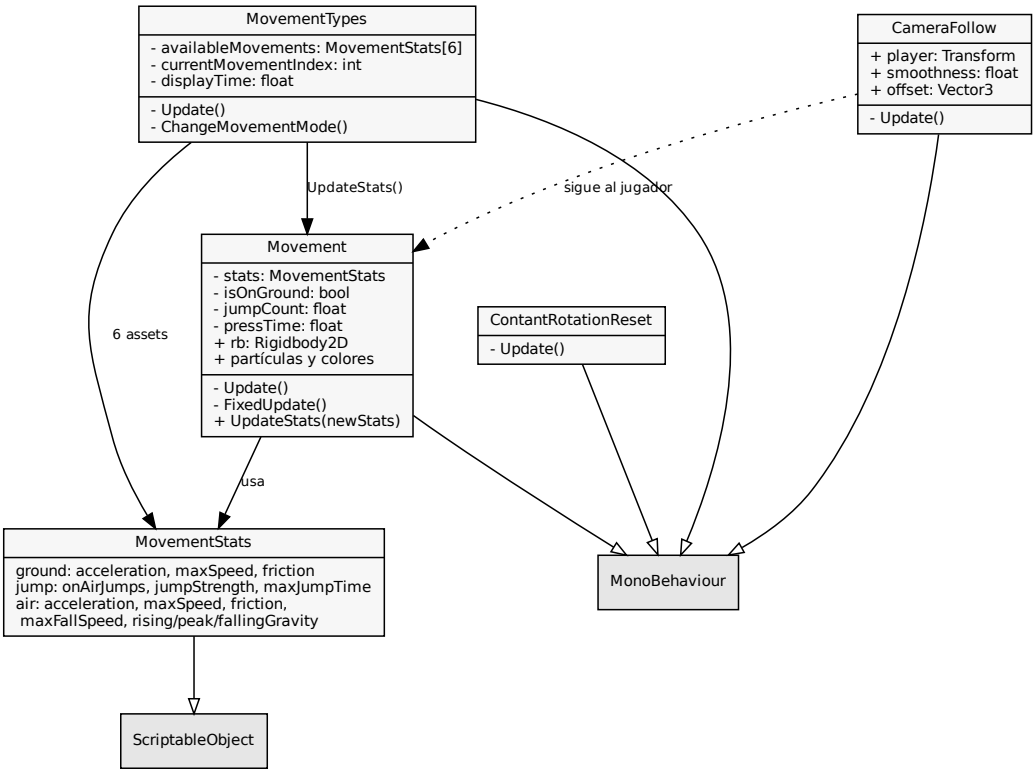


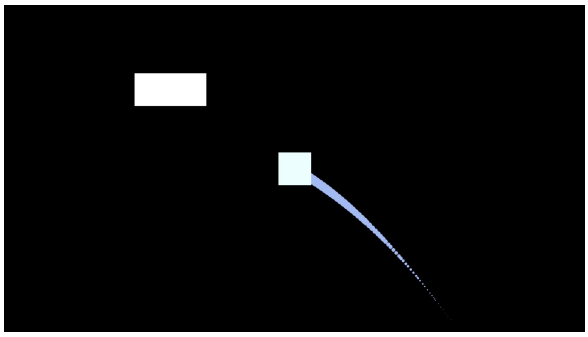
Figura 1: Diagrama de clases de Multi-Move

3 Diseño

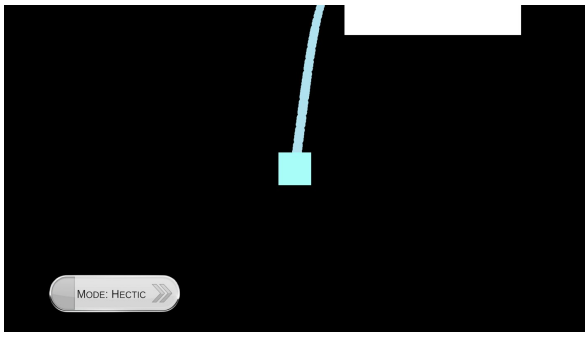
Es una escena 2D minimalista: fondo negro, plataformas blancas y el cubo del jugador. Toda la información está en el color del cubo y en el rastro de partículas que deja al moverse. Al cambiar de modo aparece unos segundos un cartel con su nombre (*Mode: Glider*).

3.1 Controles

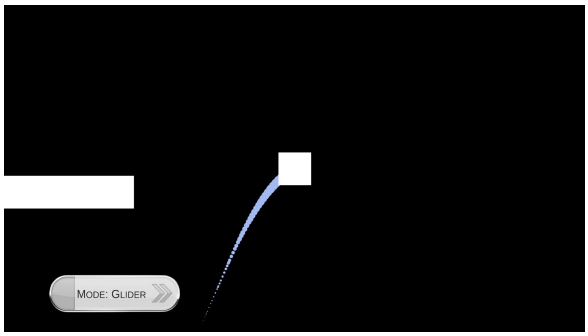
- **A / D** o las **flechas**: moverse a izquierda y derecha.
- **W**: saltar. Cuanto más tiempo se mantiene pulsada, más alto llega el salto (hasta `maxJumpTime`).
- **M**: cambiar al siguiente modo de movimiento.



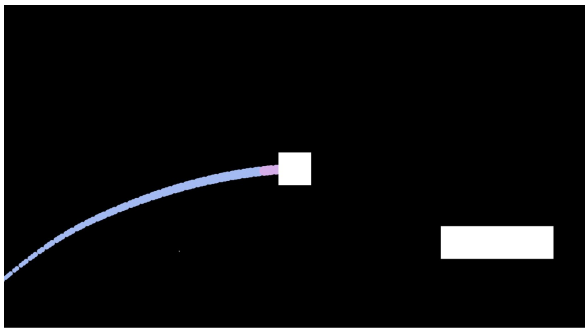
Salto con el rastro de partículas



Cartel al cambiar a Hectic



Cartel al cambiar a Glider



Caída con las partículas de color

Figura 2: Capturas de Multi-Move

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	El cubo se debe mover a izquierda y derecha con **A/D** o las flechas, con aceleración y fricción distintas en el suelo y en el aire.
RF02	El salto debe ser más alto cuanto más tiempo se mantiene pulsada la **W**.
RF03	El cubo debe poder saltar en el aire tantas veces como indique su modo.
RF04	La gravedad debe cambiar al **subir**, en el **punto más alto** y al **caer**, y la velocidad de caída debe tener un máximo.
RF05	La tecla **M** debe pasar al siguiente de los **seis modos** y mostrar su nombre durante 2 segundos.
RF06	El color del cubo debe indicar los saltos usados y el de las partículas, la fase del salto.
RF07	Debe haber partículas al saltar, al aterrizar y al intentar saltar sin saltos disponibles.
RF08	La cámara debe seguir al jugador con un movimiento suave.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	El juego debe estar hecho en **Unity** con scripts en **C#**.
RNF02	Cada modo de movimiento debe ser un **ScriptableObject** configurable desde el editor.

ID	Requisito no funcional
RNF03	La lectura del teclado se hace en `Update` y las físicas en `FixedUpdate`.
RNF04	El suelo se detecta con un **Raycast** hacia abajo contra una capa concreta.
RNF05	Debe poder jugarse en el navegador (versión WebGL en itch.io).
RNF06	Los scripts deben tener comentarios de documentación `///` para Doxygen.

5 Documentación técnica

Este anexo agrupa la documentación por scripts, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `using` y los comentarios `///`, que repiten la descripción.

5.1 MovementStats.cs

5.1.1 Ubicación

`Assets/MovementStats.cs`

Descripción general:
ScriptableObject con todos los valores de un modo de movimiento. Con `[CreateAssetMenu]` aparece en el menú *Create* del editor.

5.1.2 Funciones

MovementStats **Descripción:** Los valores se agrupan en suelo, salto y aire.
Código comentado:

```
[CreateAssetMenu]
public class MovementStats : ScriptableObject
{
    [Header("Ground")]
    public float groundAcceleration;
    public float maxGroundHorizontalSpeed;
    public float groundHorizontalFriction;

    [Header("Jump")]
    public float onAirJumps;            // saltos extra en el aire
    public float jumpStrength;
    public float maxJumpTime;           // tiempo máximo con la W pulsada

    [Header("Air")]
    public float airAcceleration;
    public float maxAirHorizontalSpeed;
    public float yVelocityLowGravityThreshold;   // margen para el "pico" del salto
    public float maxFallSpeed;
    public float airFriccion;
    public float risingtGravity;
    public float peakGravity;
    public float fallingGravity;
}
```

5.2 Movement.cs

5.2.1 Ubicación

`Assets/Movement.cs`

Descripción general:
Mueve al jugador. En `Update` lee el teclado, comprueba el suelo y cambia gravedad y colores; en `FixedUpdate` aplica las velocidades al `Rigidbody2D`.

5.2.2 Funciones

Update (dirección y suelo) **Descripción:** Calcula la dirección con las teclas y lanza un rayo hacia abajo para saber si está en el suelo. Al aterrizar suenan las partículas de choque.

Código comentado:

```
direction = Vector2.zero;
if (Input.GetKey(KeyCode.D) || Input.GetKey(KeyCode.RightArrow))
    direction += Vector2.right;
if (Input.GetKey(KeyCode.A) || Input.GetKey(KeyCode.LeftArrow))
    direction += Vector2.left;

isOnGround = Physics2D.Raycast(groundCheckOrigin.position, Vector2.down,
                               groundCheckDistance, groundLayer);

if (!wasOnGroundPreviousFrame && isOnGround)   // acaba de aterrizar
    collisionParticles.Play();
```

Update (salto) **Descripción:** Al pulsar W salta si está en el suelo o le quedan saltos en el aire, y colorea el cubo según los saltos usados. Mientras se mantiene, el salto sigue empujando.

Código comentado:

```
if (Input.GetKeyDown(KeyCode.W))
{
    if (isOnGround || jumpCount <= stats.onAirJumps)
    {
        if (isOnGround)
            jumpCount = 0;
        else
            jumpCount++;
        spriteRenderer.color = onAirColors.Evaluate(jumpCount / stats.onAirJumps);
        jumpParticles.Play();
    }
    else
    {
        noJumpParticles.Play();              // no quedan saltos
    }
}

if (Input.GetKey(KeyCode.W) && pressTime < stats.maxJumpTime && jumpCount <= stats.
    onAirJumps)
{
    pressTime += Time.deltaTime;             // salto variable
    isJumping = true;
}
if (Input.GetKeyUp(KeyCode.W))
    pressTime = 0;
```

Update (gravedad por fases) **Descripción:** Según la velocidad vertical usa la gravedad de subida, de caída o del punto más alto, y cambia el color de las partículas.

Código comentado:

```
if (rb.velocity.y > stats.yVelocityLowGravityThreshold)
{
    mainModule.startColor = risingColor;
    rb.gravityScale = stats.risingtGravity;          // subiendo
}
else if (rb.velocity.y < -stats.yVelocityLowGravityThreshold)
{
    mainModule.startColor = fallingColor;
    rb.gravityScale = stats.fallingGravity;          // cayendo
}
else if (!isOnGround)
{
    mainModule.startColor = peakColor;
    rb.gravityScale = stats.peakGravity;             // punto más alto
}
wasOnGroundPreviousFrame = isOnGround;
```

FixedUpdate **Descripción:** Acelera hasta la velocidad máxima (de suelo o de aire), aplica el salto y la fricción, y limita la velocidad de caída. Se muestra la parte del suelo; la del aire es igual con los valores de aire.

Código comentado:

```
if (isOnGround)
{
    if (rb.velocity.x < stats.maxGroundHorizontalSpeed && rb.velocity.x > -stats.
        maxGroundHorizontalSpeed)
    {
        rb.velocity = new Vector2(rb.velocity.x + (direction.x * stats.
            groundAcceleration * Time.deltaTime), rb.velocity.y);
    }
    else
    {
        // si supera la velocidad máxima, se limita
        if (rb.velocity.x > stats.maxGroundHorizontalSpeed)
            rb.velocity = new Vector2(stats.maxGroundHorizontalSpeed, rb.velocity.y);
        else if (rb.velocity.x < -stats.maxGroundHorizontalSpeed)
            rb.velocity = new Vector2(-stats.maxGroundHorizontalSpeed, rb.velocity.y);
    }
}

if (isJumping)
{
    rb.velocity = new Vector2(rb.velocity.x, stats.jumpStrength);
    isJumping = false;
}

if (isOnGround)
{
    // la fricción frena hacia 0 en los dos sentidos
    if (rb.velocity.x > 0)
        rb.velocity = new Vector2(rb.velocity.x - stats.groundHorizontalFriction *
            Time.deltaTime, rb.velocity.y);
    else if (rb.velocity.x < 0)
        rb.velocity = new Vector2(rb.velocity.x + stats.groundHorizontalFriction *
            Time.deltaTime, rb.velocity.y);
}

if (rb.velocity.y < -stats.maxFallSpeed)
    rb.velocity = new Vector2(rb.velocity.x, -stats.maxFallSpeed);
```

UpdateStats **Descripción:** Cambia el modo activo; lo llama `MovementTypes`.

Código comentado:

```
public void UpdateStats(MovementStats newStats)
{
    stats = newStats;
}
```

5.3 MovementTypes.cs

5.3.1 Ubicación

`Assets/MovementTypes.cs`

Descripción general:
Guarda la lista de modos y los va cambiando con la tecla M.

5.3.2 Funciones

ChangeMovementMode **Descripción:** Pasa al siguiente modo (vuelve al primero después del último), muestra su nombre 2 segundos y se lo pasa al jugador.

Código comentado:

```
private void ChangeMovementMode()
{
    currentMovementIndex++;
    if (currentMovementIndex >= availableMovements.Length)
        currentMovementIndex = 0;

    currentStats = availableMovements[currentMovementIndex];
    movementText.text = "Mode: " + currentStats.name;   // nombre del asset
    movementImage.gameObject.SetActive(true);
    movementText.gameObject.SetActive(true);
    displayTime = 2f;

    FindObjectOfType<Movement>().UpdateStats(currentStats);
}
```

5.4 CameraFollow.cs y ContantRotationReset.cs

5.4.1 Ubicación

`Assets`

Descripción general:
La cámara sigue al jugador y el cubo no gira aunque choque.

5.4.2 Funciones

CameraFollow.Update **Descripción:** Se acerca poco a poco a la posición del jugador más el desfase con `SmoothDamp`.
Código comentado:

```
void Update()
{
    Vector3 targetPosition = player.position + offset;
    transform.position = Vector3.SmoothDamp(transform.position, targetPosition,
                                            ref velocity, smoothness);
}
```

ContantRotationReset.Update **Descripción:** Deja la rotación a cero en cada fotograma.
Código comentado:

```
void Update()
{
    transform.rotation = Quaternion.identity;
}
```